Universidad de Cundinamarca

Especialización en Analítica y Ciencia de Datos

Informe de práctica R2-A2-S12

Ejercicios resueltos de bases de datos NewSQL y NoSQL

Asignatura: Big Data (604025)

Estudiante: Nikolas Krejci

Fecha: 25 de octubre de 2026

Repositorio: https://github.com/Migatte-Yk/NoSqlNewSqlBigData

1. Resumen

Se desarrolló un caso de estudio de gestión de estudiantes y matrículas con datos sintéticos (5 programas, 24 estudiantes y 36 matrículas) sobre dos motores: CockroachDB v26.3.2, como motor NewSQL, y MongoDB 9.0.2, como motor NoSQL documental, administrado con mongosh y MongoDB Compass. Se ejecutaron scripts SQL y JavaScript para verificar los motores, crear el esquema, cargar datos, consultar, modificar, agregar y probar transacciones. Los resultados de las agregaciones coincidieron entre ambos motores. Ambos se usaron en un solo nodo local, por lo que no se evaluaron distribución, alta disponibilidad ni rendimiento. La exportación de datos desde Compass se documentó pero no se ejecutó.

2. Introducción

Este informe corresponde a la actividad R2-A2-S12 de la asignatura Big Data. El docente proporcionó la guía «Guía práctica: Instalación y configuración de NewSQL», que indica usar CockroachDB para NewSQL y MongoDB con Compass o línea de comandos para NoSQL, pero no incluyó una batería de ejercicios. Por ello, los ejercicios aquí presentados fueron diseñados por el estudiante a partir de los requisitos de la actividad en Moodle, y no corresponden a ejercicios establecidos oficialmente por el docente.

3. Objetivo general

Resolver y documentar ejercicios prácticos de bases de datos con un motor NewSQL y uno NoSQL documental, usando consultas y comandos SQL y NoSQL sobre un mismo caso de estudio.

4. Objetivos específicos

- Verificar el entorno de CockroachDB y construir, cargar y consultar un esquema relacional con restricciones de integridad y transacciones.
- Instalar MongoDB Community Server, mongosh y Compass, y modelar el mismo caso como documentos, con consultas, actualizaciones y agregaciones.
- Comparar ambos enfoques, distinguiendo lo que indica la documentación de lo observado durante la práctica.

5. Marco conceptual

Big Data. El NIST Big Data Public Working Group (2015) dedica el volumen 1 de su marco de interoperabilidad a definir el término y los conceptos asociados. El concepto se refiere a

conjuntos de datos extensos, caracterizados por su volumen, variedad, velocidad y variabilidad, que requieren arquitecturas escalables para almacenarse, manipularse y analizarse.

NewSQL. Pavlo y Aslett (2016) describen NewSQL como una clase de sistemas que afirman escalar cargas OLTP más allá de los sistemas relacionales tradicionales, y cuestionan si esa superioridad es real o en parte publicidad. CockroachDB se presenta en Taft et al. (2020) como un SGBD SQL escalable, diseñado para cargas OLTP globales con alta disponibilidad y consistencia fuerte. Estas son afirmaciones de los autores; en esta práctica no se probaron ni la escalabilidad ni la alta disponibilidad.

Transacciones en CockroachDB. Según su documentación, las transacciones agrupan sentencias en una unidad de todo o nada: `BEGIN` las inicia, `COMMIT` las confirma y `ROLLBACK` las aborta; por defecto el aislamiento es SERIALIZABLE (Cockroach Labs, s. f.-a, s. f.-b).

NoSQL. Cattell (2010) revisa almacenes SQL y NoSQL diseñados para cargas simples distribuidas en muchos servidores y los compara por modelo de datos, consistencia, durabilidad, disponibilidad y capacidad de consulta; señala que suelen sacrificar propiedades como la consistencia transaccional global a cambio de disponibilidad y escalabilidad. MongoDB es un almacén documental; su manual indica que, para muchos escenarios, el modelo desnormalizado (documentos y arrays embebidos) sigue siendo óptimo (MongoDB, Inc., s. f.-b), y que el análisis de datos se expresa mediante pipelines de etapas (MongoDB, Inc., s. f.-a).

6. Herramientas y entorno de ejecución

Componente	Versión / modo detectado
Sistema operativo	Windows 11 Pro
CockroachDB	v26.3.2 (CCL); `start-single-node --insecure`, un nodo, desarrollo local, sin TLS; interfaz web en `localhost:8080`
MongoDB Community Server	9.0.2, instancia standalone en `localhost:27017`, servicio de Windows
mongosh	2.14.0
MongoDB Compass	1.53.1
Git / Python	Git 2.53.0 / Python 3.14 (generador de datos y del PDF)

CockroachDB ya estaba instalado y con un almacén de datos previo; se reutilizó esa instancia sin crear otro clúster. MongoDB, mongosh y Compass se instalaron con `winget` (paquetes oficiales `MongoDB.Server`, `MongoDB.Shell` y `MongoDB.Compass.Full`); el servidor y Compass son instalaciones separadas.

7. Metodología

1. **Preparación y recolección de datos (dentro del entorno experimental).** Los datos son 100 % sintéticos: nombres y correos ficticios (dominio `.invalid`). Se generaron de forma determinista con `datos/generar_datos.py`, que produce `datos/programas.json`,

`datos/estudiantes.json` (con las matrículas embebidas) y el script SQL de carga. De esta forma ambos motores contienen los mismos registros lógicos y la carga puede repetirse sin servicios externos. No hubo recolección desde fuentes externas.

2. **Ejecución.** Los scripts de `cockroachdb/` se ejecutaron con `cockroach sql -f` y los de `mongodb/` con `mongosh --file`.
3. **Registro.** Las salidas de texto están en `evidencias/salidas/` y las capturas de pantalla en `evidencias/`.
4. **Reinicialización.** Los scripts `02_esquema.sql`, `03_carga_datos.sql` y `02_carga_datos.js` solo eliminan y recrean objetos del proyecto (base `practica_matriculas`).

8. Desarrollo de la práctica con CockroachDB

Todos los scripts se encuentran en `cockroachdb/`.

A1. Verificación del motor

Objetivo. Identificar versión y modo de ejecución, verificar la conexión y comprobar el contexto de la sesión.

Procedimiento. `cockroach sql --insecure --host=localhost:26257 -f cockroachdb\01_verificacion.sql`. El script consulta `version()`, el usuario, la base y la zona horaria actuales, la versión del clúster y las bases existentes.

```
SELECT version();
SELECT current_user() AS usuario, current_database() AS base_actual,
current_setting('TIMEZONE') AS zona_horaria;
SHOW CLUSTER SETTING version;
```

Evidencia.

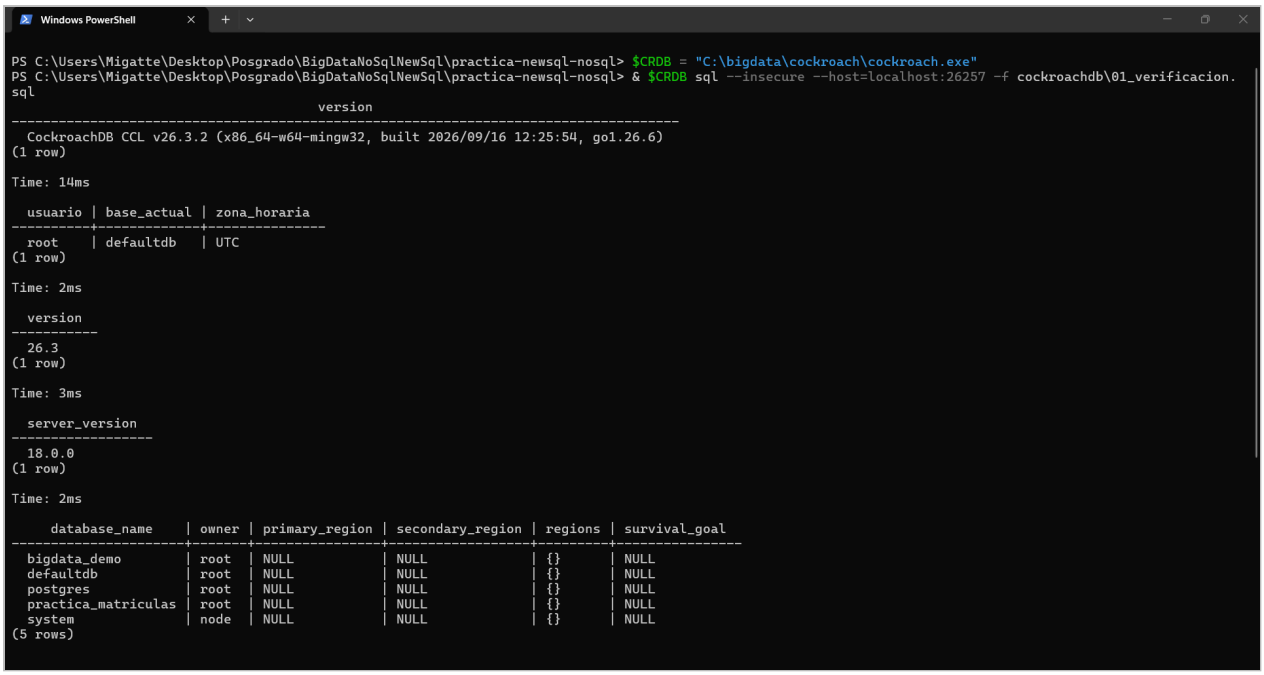


Figura 1. Versión de CockroachDB y contexto de la sesión (A1).

Resultado observado. CockroachDB CCL v26.3.2; usuario `root`, base `defaultdb`, zona horaria UTC; versión de clúster 26.3. La lista de bases ya incluye `practica_matriculas`. `SHOW server_version` devuelve 18.0.0, que corresponde a la versión de compatibilidad con PostgreSQL y no a la de CockroachDB.

Interpretación técnica. El motor se ejecuta como un único nodo en modo inseguro (desarrollo local). Esto es suficiente para practicar SQL, pero no constituye un clúster de producción ni permite observar distribución física entre nodos. En la documentación, CockroachDB se describe como un SGBD SQL distribuido con consistencia fuerte (Taft et al., 2020); aquí solo se verificó su comportamiento SQL en un nodo.

A2. Diseño del esquema

Objetivo. Crear una base dedicada con tablas de programas, estudiantes y matrículas, con claves y restricciones.

Procedimiento. `cockroachdb\02_esquema.sql` crea `practica_matriculas` y tres tablas: `programas` (PK `programa_id`, `codigo` único, CHECK de duración), `estudiantes` (PK `codigo`, FK a `programas`, `correo` único, `telefono` opcional) y `matriculas` (PK con identidad, FK a `estudiantes`, `UNIQUE (codigo_estudiante, semestre)`, CHECK sobre semestre, estado, créditos y valor).

```
CREATE TABLE matriculas (
    matricula_id      INT8 PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
    codigo_estudiante STRING(10) NOT NULL REFERENCES estudiantes (codigo),
    semestre          STRING(6)  NOT NULL CHECK (semestre ~ '^[0-9]{4}-[12]$'),
    estado            STRING     NOT NULL CHECK (estado IN
('ACTIVA','FINALIZADA','CANCELADA','APLAZADA')),
    CONSTRAINT uq_est_semestre UNIQUE (codigo_estudiante, semestre) ...
);
```

Evidencia.

```
Time: 121ms

  schema_name |         table_name          |   type   | owner | estimated_row_count | locality
--------------+-----------------------------+----------+-------+---------------------+-----------
  public      | estudiantes                 | table    | root  |                   0 | NULL
  public      | matriculas                  | table    | root  |                   0 | NULL
  public      | matriculas_matricula_id_seq | sequence | root  |                   0 | NULL
  public      | programas                   | table    | root  |                   0 | NULL
(4 rows)

Time: 50ms

  table_name |                                                  create_statement
-------------+---------------------------------------------------------------------------------------------------------------------------------------------------------
  matriculas | CREATE TABLE public.matriculas (
             |     matricula_id INT8 NOT NULL GENERATED BY DEFAULT AS IDENTITY,
             |     codigo_estudiante STRING(10) NOT NULL,
             |     semestre STRING(6) NOT NULL,
             |     fecha_matricula DATE NOT NULL,
             |     estado STRING NOT NULL,
             |     creditos INT2 NOT NULL,
             |     valor DECIMAL(12,2) NOT NULL,
             |     CONSTRAINT matriculas_pkey PRIMARY KEY (matricula_id ASC),
             |     CONSTRAINT matriculas_codigo_estudiante_fkey FOREIGN KEY (codigo_estudiante) REFERENCES public.estudiantes(codigo),
             |     UNIQUE INDEX uq_est_semestre (codigo_estudiante ASC, semestre ASC),
             |     CONSTRAINT check_semestre CHECK (semestre ~ '^[0-9]{4}-[12]$':::STRING),
             |     CONSTRAINT check_estado CHECK (estado IN ('ACTIVA':::STRING, 'FINALIZADA':::STRING, 'CANCELADA':::STRING, 'APLAZADA':::STRING)),
             |     CONSTRAINT check_creditos CHECK (creditos BETWEEN 1:::INT8 AND 24:::INT8),
             |     CONSTRAINT check_valor CHECK (valor >= 0:::DECIMAL)
             | ) WITH (schema_locked = true);
(1 row)

Time: 38ms

  table_name  |           constraint_name            | constraint_type |                          details                          | validated
--------------+--------------------------------------+-----------------+-----------------------------------------------------------+------------
  estudiantes | ck_correo                            | CHECK           | CHECK ((correo LIKE '%@%'::text))                         |     t
  estudiantes | estudiantes_apellidos_not_null       | NOT NULL        | NOT NULL                                                  |     t
  estudiantes | estudiantes_ciudad_not_null          | NOT NULL        | NOT NULL                                                  |     t
  estudiantes | estudiantes_codigo_not_null          | NOT NULL        | NOT NULL                                                  |     t
  estudiantes | estudiantes_correo_key               | UNIQUE          | UNIQUE (correo ASC)                                       |     t
  estudiantes | estudiantes_correo_not_null          | NOT NULL        | NOT NULL                                                  |     t
  estudiantes | estudiantes_fecha_ingreso_not_null   | NOT NULL        | NOT NULL                                                  |     t
  estudiantes | estudiantes_fecha_nacimiento_not_null | NOT NULL        | NOT NULL                                                  |     t
  estudiantes | estudiantes_nombres_not_null         | NOT NULL        | NOT NULL                                                  |     t
```

Figura 2. Tablas creadas, definición de matriculas y restricciones (A2).

Resultado observado. `SHOW TABLES` lista `programas`, `estudiantes`, `matriculas` y la secuencia de identidad. `SHOW CREATE TABLE` y `SHOW CONSTRAINTS` muestran las claves primarias y foráneas y las restricciones CHECK, UNIQUE y NOT NULL.

Interpretación técnica. El esquema impone la integridad en el propio motor (dominio de estados, unicidad por semestre, referencias entre tablas), lo que se aprovecha en A5.

A3. Inserción y carga

Objetivo. Cargar los datos de forma repetible y comprobar las cantidades.

Procedimiento. `cockroachdb\03_carga_datos.sql` (generado por `datos/generar_datos.py`) vacía solo las tres tablas del proyecto, de hijas a padres, e inserta programas, estudiantes y matrículas; termina con un conteo.

Evidencia.



Figura 3. Registros cargados y conteos (A3).

Resultado observado. `INSERT 0 5`, `INSERT 0 24` e `INSERT 0 36`; conteos: 5 programas, 24 estudiantes, 36 matrículas.

Interpretación técnica. Al ser determinista, la carga puede repetirse y deja siempre el mismo estado inicial. Debe usarse únicamente para reiniciar la práctica, porque elimina los datos de las tres tablas.

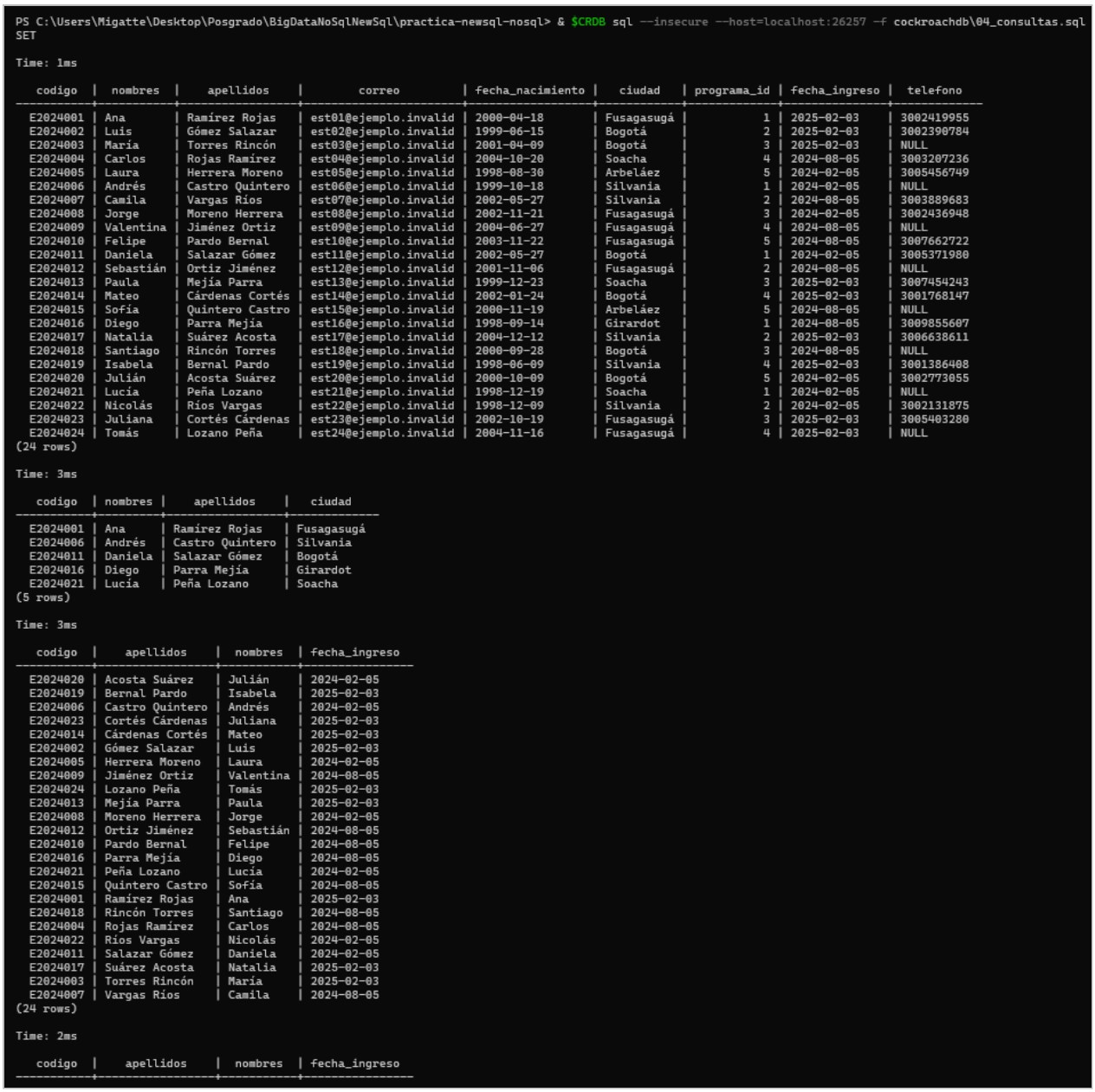
A4. Consultas SQL

Objetivo. Practicar consultas generales, filtros, ordenamiento, JOIN, agrupaciones, HAVING y subconsultas.

Procedimiento. `cockroachdb\04_consultas.sql` contiene diez consultas, cada una con un comentario que explica su objetivo. Fragmento (consulta 6):

```
SELECT p.nombre AS programa, count(e.codigo) AS total_estudiantes
FROM programas p LEFT JOIN estudiantes e ON e.programa_id = p.programa_id
GROUP BY p.nombre ORDER BY total_estudiantes DESC, p.nombre;
```

Evidencia.



```
PS C:\Users\Migatte\Desktop\Posgrado\BigDataNoSqlNewSql\practica-newsql-nosql> & $CRDB sql --insecure --host=localhost:26257 -f cockroachdb\04_consultas.sql
SET

Time: 1ms

   codigo  |  nombres  |    apellidos    |        correo         | fecha_nacimiento |   ciudad    | programa_id | fecha_ingreso |  telefono
-----------+-----------+-----------------+-----------------------+------------------+-------------+-------------+---------------+-------------
  E2024001 | Ana       | Ramírez Rojas   | est01@ejemplo.invalid | 2000-04-18       | Fusagasugá  |           1 | 2025-02-03    | 3002419955
  E2024002 | Luis      | Gómez Salazar   | est02@ejemplo.invalid | 1999-06-15       | Bogotá      |           2 | 2025-02-03    | 3002390784
  E2024003 | María     | Torres Rincón   | est03@ejemplo.invalid | 2001-04-09       | Bogotá      |           3 | 2025-02-03    | NULL
  E2024004 | Carlos    | Rojas Ramírez   | est04@ejemplo.invalid | 2004-10-20       | Soacha      |           4 | 2024-08-05    | 3003207236
  E2024005 | Laura     | Herrera Moreno  | est05@ejemplo.invalid | 1998-08-30       | Arbeláez    |           5 | 2024-02-05    | 3005456749
  E2024006 | Andrés    | Castro Quintero | est06@ejemplo.invalid | 1999-10-18       | Silvania    |           1 | 2024-02-05    | NULL
  E2024007 | Camila    | Vargas Ríos     | est07@ejemplo.invalid | 2002-05-27       | Silvania    |           2 | 2024-08-05    | 3003889683
  E2024008 | Jorge     | Moreno Herrera  | est08@ejemplo.invalid | 2002-11-21       | Fusagasugá  |           3 | 2024-02-05    | 3002436948
  E2024009 | Valentina | Jiménez Ortiz   | est09@ejemplo.invalid | 2004-06-27       | Fusagasugá  |           4 | 2024-08-05    | NULL
  E2024010 | Felipe    | Pardo Bernal    | est10@ejemplo.invalid | 2003-11-22       | Fusagasugá  |           5 | 2024-08-05    | 3007662722
  E2024011 | Daniela   | Salazar Gómez   | est11@ejemplo.invalid | 2002-05-27       | Bogotá      |           1 | 2024-02-05    | 3005371980
  E2024012 | Sebastián | Ortiz Jiménez   | est12@ejemplo.invalid | 2001-11-06       | Fusagasugá  |           2 | 2024-08-05    | NULL
  E2024013 | Paula     | Mejía Parra     | est13@ejemplo.invalid | 1999-12-23       | Soacha      |           3 | 2025-02-03    | 3007454243
  E2024014 | Mateo     | Cárdenas Cortés | est14@ejemplo.invalid | 2002-01-24       | Bogotá      |           4 | 2025-02-03    | 3001768147
  E2024015 | Sofía     | Quintero Castro | est15@ejemplo.invalid | 2000-11-19       | Arbeláez    |           5 | 2024-08-05    | NULL
  E2024016 | Diego     | Parra Mejía     | est16@ejemplo.invalid | 1998-09-14       | Girardot    |           1 | 2024-08-05    | 3009855607
  E2024017 | Natalia   | Suárez Acosta   | est17@ejemplo.invalid | 2004-12-12       | Silvania    |           2 | 2025-02-03    | 3006638611
  E2024018 | Santiago  | Rincón Torres   | est18@ejemplo.invalid | 2000-09-28       | Bogotá      |           3 | 2024-08-05    | NULL
  E2024019 | Isabela   | Bernal Pardo    | est19@ejemplo.invalid | 1998-06-09       | Silvania    |           4 | 2025-02-03    | 3001386408
  E2024020 | Julián    | Acosta Suárez   | est20@ejemplo.invalid | 2000-10-09       | Bogotá      |           5 | 2024-02-05    | 3002773055
  E2024021 | Lucía     | Peña Lozano     | est21@ejemplo.invalid | 1998-12-19       | Soacha      |           1 | 2024-02-05    | NULL
  E2024022 | Nicolás   | Ríos Vargas     | est22@ejemplo.invalid | 1998-12-09       | Silvania    |           2 | 2024-02-05    | 3002131875
  E2024023 | Juliana   | Cortés Cárdenas | est23@ejemplo.invalid | 2002-10-19       | Fusagasugá  |           3 | 2025-02-03    | 3005403280
  E2024024 | Tomás     | Lozano Peña     | est24@ejemplo.invalid | 2004-11-16       | Fusagasugá  |           4 | 2025-02-03    | NULL
(24 rows)

Time: 3ms

   codigo  | nombres |    apellidos    |   ciudad
-----------+---------+-----------------+-------------
  E2024001 | Ana     | Ramírez Rojas   | Fusagasugá
  E2024006 | Andrés  | Castro Quintero | Silvania
  E2024011 | Daniela | Salazar Gómez   | Bogotá
  E2024016 | Diego   | Parra Mejía     | Girardot
  E2024021 | Lucía   | Peña Lozano     | Soacha
(5 rows)

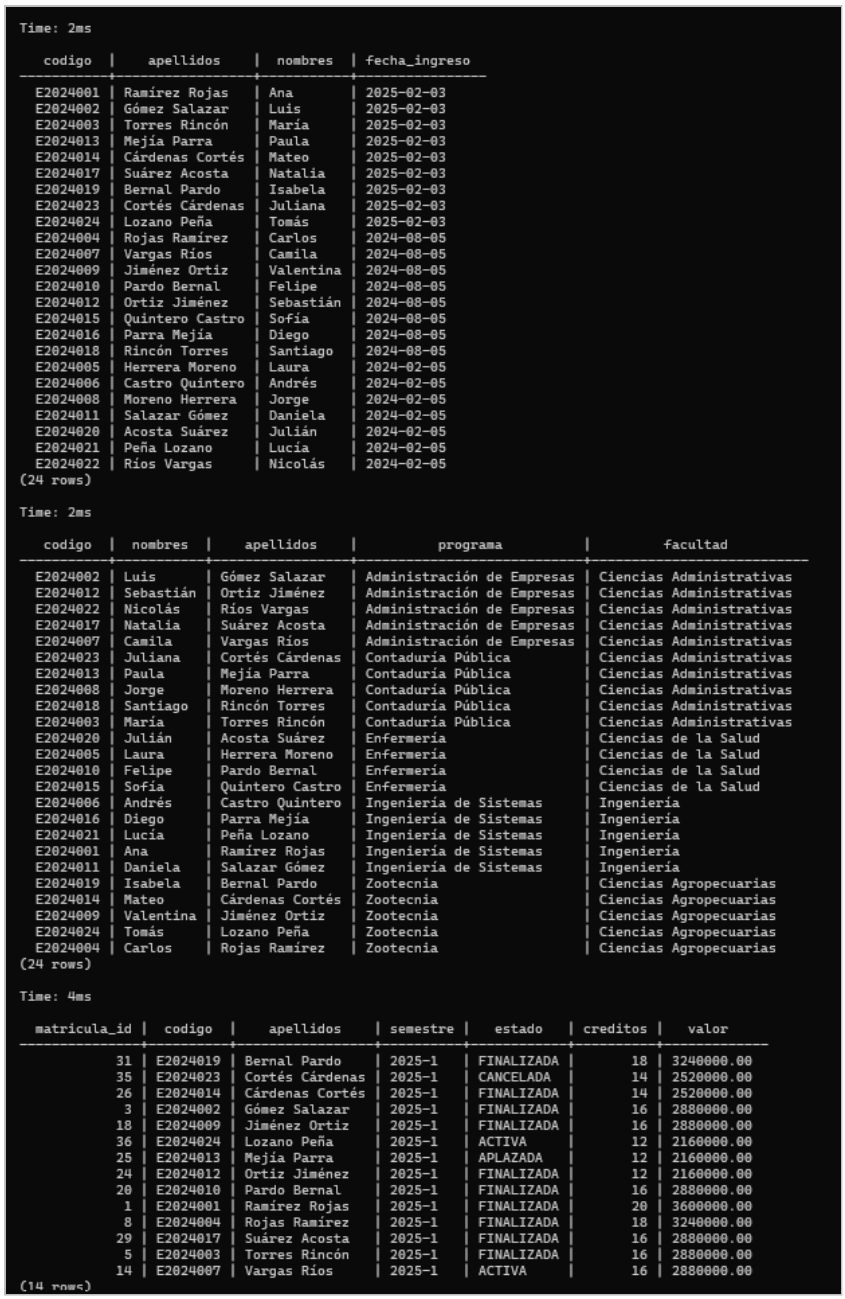
Time: 3ms

   codigo  |    apellidos    |  nombres  | fecha_ingreso
-----------+-----------------+-----------+---------------
  E2024020 | Acosta Suárez   | Julián    | 2024-02-05
  E2024019 | Bernal Pardo    | Isabela   | 2025-02-03
  E2024006 | Castro Quintero | Andrés    | 2024-02-05
  E2024023 | Cortés Cárdenas | Juliana   | 2025-02-03
  E2024014 | Cárdenas Cortés | Mateo     | 2025-02-03
  E2024002 | Gómez Salazar   | Luis      | 2025-02-03
  E2024005 | Herrera Moreno  | Laura     | 2024-02-05
  E2024009 | Jiménez Ortiz   | Valentina | 2024-08-05
  E2024024 | Lozano Peña     | Tomás     | 2025-02-03
  E2024013 | Mejía Parra     | Paula     | 2025-02-03
  E2024008 | Moreno Herrera  | Jorge     | 2024-02-05
  E2024012 | Ortiz Jiménez   | Sebastián | 2024-08-05
  E2024010 | Pardo Bernal    | Felipe    | 2024-08-05
  E2024016 | Parra Mejía     | Diego     | 2024-08-05
  E2024021 | Peña Lozano     | Lucía     | 2024-02-05
  E2024015 | Quintero Castro | Sofía     | 2024-08-05
  E2024001 | Ramírez Rojas   | Ana       | 2025-02-03
  E2024018 | Rincón Torres   | Santiago  | 2024-08-05
  E2024004 | Rojas Ramírez   | Carlos    | 2024-08-05
  E2024022 | Ríos Vargas     | Nicolás   | 2024-02-05
  E2024011 | Salazar Gómez   | Daniela   | 2024-02-05
  E2024017 | Suárez Acosta   | Natalia   | 2025-02-03
  E2024003 | Torres Rincón   | María     | 2025-02-03
  E2024007 | Vargas Ríos     | Camila    | 2024-08-05
(24 rows)

Time: 2ms

   codigo  |    apellidos    |  nombres  | fecha_ingreso
-----------+-----------------+-----------+---------------
```

Figura 4. Consultas 1 a 3: listado general, filtro por programa y ordenamientos (A4).



```
Time: 2ms

   codigo   |    apellidos    |  nombres  | fecha_ingreso
------------+-----------------+-----------+---------------
  E2024001  | Ramírez Rojas   | Ana       | 2025-02-03
  E2024002  | Gómez Salazar   | Luis      | 2025-02-03
  E2024003  | Torres Rincón   | María     | 2025-02-03
  E2024013  | Mejía Parra     | Paula     | 2025-02-03
  E2024014  | Cárdenas Cortés | Mateo     | 2025-02-03
  E2024017  | Suárez Acosta   | Natalia   | 2025-02-03
  E2024019  | Bernal Pardo    | Isabela   | 2025-02-03
  E2024023  | Cortés Cárdenas | Juliana   | 2025-02-03
  E2024024  | Lozano Peña     | Tomás     | 2025-02-03
  E2024004  | Rojas Ramírez   | Carlos    | 2024-08-05
  E2024007  | Vargas Ríos     | Camila    | 2024-08-05
  E2024009  | Jiménez Ortiz   | Valentina | 2024-08-05
  E2024010  | Pardo Bernal    | Felipe    | 2024-08-05
  E2024012  | Ortiz Jiménez   | Sebastián | 2024-08-05
  E2024015  | Quintero Castro | Sofía     | 2024-08-05
  E2024016  | Parra Mejía     | Diego     | 2024-08-05
  E2024018  | Rincón Torres   | Santiago  | 2024-08-05
  E2024005  | Herrera Moreno  | Laura     | 2024-02-05
  E2024006  | Castro Quintero | Andrés    | 2024-02-05
  E2024008  | Moreno Herrera  | Jorge     | 2024-02-05
  E2024011  | Salazar Gómez   | Daniela   | 2024-02-05
  E2024020  | Acosta Suárez   | Julián    | 2024-02-05
  E2024021  | Peña Lozano     | Lucía     | 2024-02-05
  E2024022  | Ríos Vargas     | Nicolás   | 2024-02-05
(24 rows)

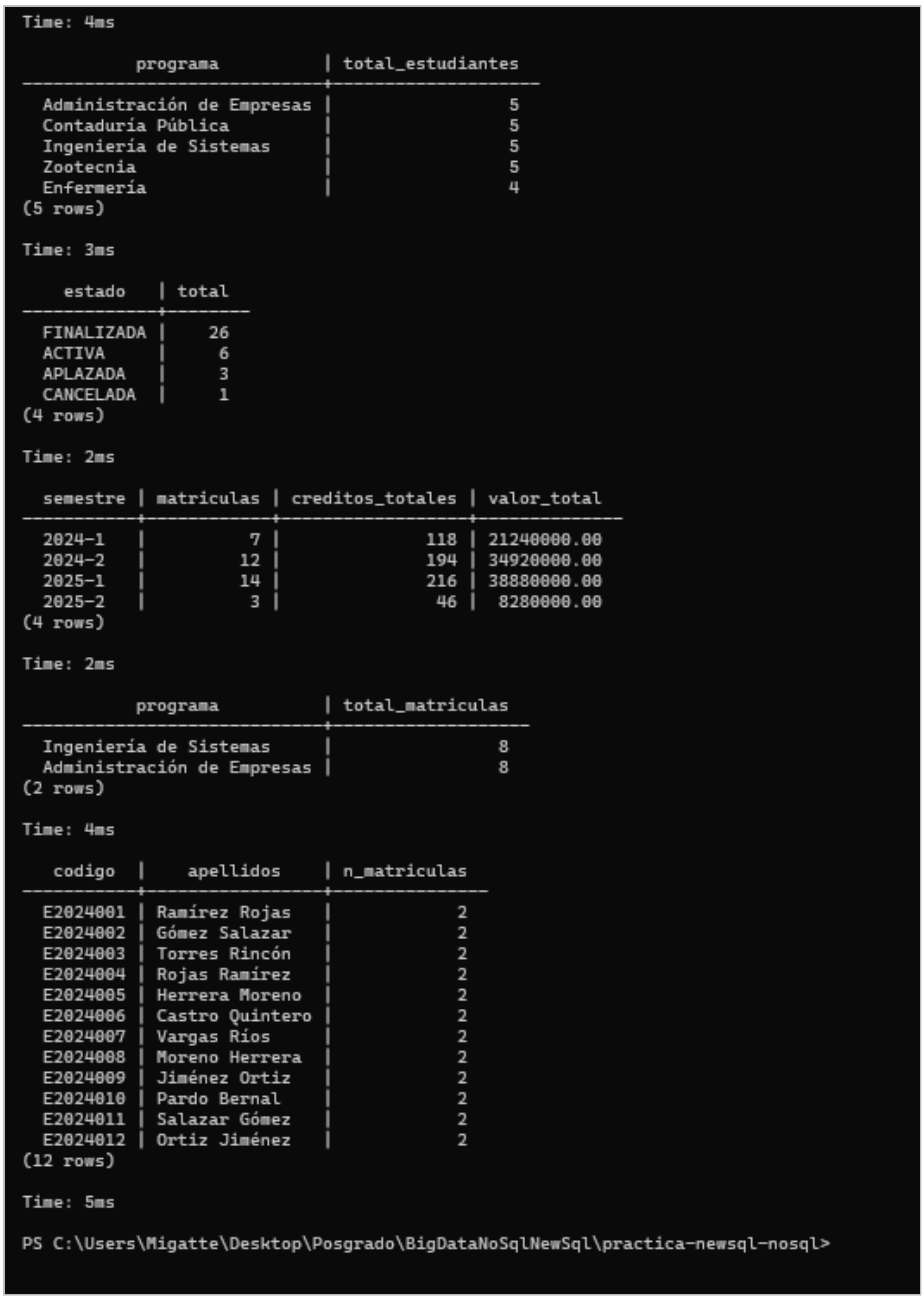
Time: 2ms

   codigo   |  nombres  |    apellidos    |          programa          |         facultad
------------+-----------+-----------------+----------------------------+--------------------------
  E2024002  | Luis      | Gómez Salazar   | Administración de Empresas | Ciencias Administrativas
  E2024012  | Sebastián | Ortiz Jiménez   | Administración de Empresas | Ciencias Administrativas
  E2024022  | Nicolás   | Ríos Vargas     | Administración de Empresas | Ciencias Administrativas
  E2024017  | Natalia   | Suárez Acosta   | Administración de Empresas | Ciencias Administrativas
  E2024007  | Camila    | Vargas Ríos     | Administración de Empresas | Ciencias Administrativas
  E2024023  | Juliana   | Cortés Cárdenas | Contaduría Pública         | Ciencias Administrativas
  E2024013  | Paula     | Mejía Parra     | Contaduría Pública         | Ciencias Administrativas
  E2024008  | Jorge     | Moreno Herrera  | Contaduría Pública         | Ciencias Administrativas
  E2024018  | Santiago  | Rincón Torres   | Contaduría Pública         | Ciencias Administrativas
  E2024003  | María     | Torres Rincón   | Contaduría Pública         | Ciencias Administrativas
  E2024020  | Julián    | Acosta Suárez   | Enfermería                 | Ciencias de la Salud
  E2024005  | Laura     | Herrera Moreno  | Enfermería                 | Ciencias de la Salud
  E2024010  | Felipe    | Pardo Bernal    | Enfermería                 | Ciencias de la Salud
  E2024015  | Sofía     | Quintero Castro | Enfermería                 | Ciencias de la Salud
  E2024006  | Andrés    | Castro Quintero | Ingeniería de Sistemas     | Ingeniería
  E2024016  | Diego     | Parra Mejía     | Ingeniería de Sistemas     | Ingeniería
  E2024021  | Lucía     | Peña Lozano     | Ingeniería de Sistemas     | Ingeniería
  E2024001  | Ana       | Ramírez Rojas   | Ingeniería de Sistemas     | Ingeniería
  E2024011  | Daniela   | Salazar Gómez   | Ingeniería de Sistemas     | Ingeniería
  E2024019  | Isabela   | Bernal Pardo    | Zootecnia                  | Ciencias Agropecuarias
  E2024014  | Mateo     | Cárdenas Cortés | Zootecnia                  | Ciencias Agropecuarias
  E2024009  | Valentina | Jiménez Ortiz   | Zootecnia                  | Ciencias Agropecuarias
  E2024024  | Tomás     | Lozano Peña     | Zootecnia                  | Ciencias Agropecuarias
  E2024004  | Carlos    | Rojas Ramírez   | Zootecnia                  | Ciencias Agropecuarias
(24 rows)

Time: 4ms

 matricula_id | codigo   |    apellidos    | semestre |   estado   | creditos |   valor
--------------+----------+-----------------+----------+------------+----------+------------
           31 | E2024019 | Bernal Pardo    | 2025-1   | FINALIZADA |       18 | 3240000.00
           35 | E2024023 | Cortés Cárdenas | 2025-1   | CANCELADA  |       14 | 2520000.00
           26 | E2024014 | Cárdenas Cortés | 2025-1   | FINALIZADA |       14 | 2520000.00
            3 | E2024002 | Gómez Salazar   | 2025-1   | FINALIZADA |       16 | 2880000.00
           18 | E2024009 | Jiménez Ortiz   | 2025-1   | FINALIZADA |       16 | 2880000.00
           36 | E2024024 | Lozano Peña     | 2025-1   | ACTIVA     |       12 | 2160000.00
           25 | E2024013 | Mejía Parra     | 2025-1   | APLAZADA   |       12 | 2160000.00
           24 | E2024012 | Ortiz Jiménez   | 2025-1   | FINALIZADA |       12 | 2160000.00
           20 | E2024010 | Pardo Bernal    | 2025-1   | FINALIZADA |       16 | 2880000.00
            1 | E2024001 | Ramírez Rojas   | 2025-1   | FINALIZADA |       20 | 3600000.00
            8 | E2024004 | Rojas Ramírez   | 2025-1   | FINALIZADA |       18 | 3240000.00
           29 | E2024017 | Suárez Acosta   | 2025-1   | FINALIZADA |       16 | 2880000.00
            5 | E2024003 | Torres Rincón   | 2025-1   | FINALIZADA |       16 | 2880000.00
           14 | E2024007 | Vargas Ríos     | 2025-1   | ACTIVA     |       16 | 2880000.00
(14 rows)
```

Figura 5. Consultas 3 a 5: ordenamiento, JOIN estudiantes–programas y matrículas del semestre 2025-1 (A4).



```
Time: 4ms

           programa          | total_estudiantes
-----------------------------+-------------------
  Administración de Empresas |                 5
  Contaduría Pública         |                 5
  Ingeniería de Sistemas     |                 5
  Zootecnia                  |                 5
  Enfermería                 |                 4
(5 rows)

Time: 3ms

    estado   | total
-------------+--------
  FINALIZADA |    26
  ACTIVA     |     6
  APLAZADA   |     3
  CANCELADA  |     1
(4 rows)

Time: 2ms

  semestre | matriculas | creditos_totales | valor_total
-----------+------------+------------------+--------------
  2024-1   |          7 |              118 | 21240000.00
  2024-2   |         12 |              194 | 34920000.00
  2025-1   |         14 |              216 | 38880000.00
  2025-2   |          3 |               46 |  8280000.00
(4 rows)

Time: 2ms

           programa          | total_matriculas
-----------------------------+------------------
  Ingeniería de Sistemas     |                8
  Administración de Empresas |                8
(2 rows)

Time: 4ms

   codigo   |    apellidos    | n_matriculas
------------+-----------------+--------------
  E2024001 | Ramírez Rojas   |            2
  E2024002 | Gómez Salazar   |            2
  E2024003 | Torres Rincón   |            2
  E2024004 | Rojas Ramírez   |            2
  E2024005 | Herrera Moreno  |            2
  E2024006 | Castro Quintero |            2
  E2024007 | Vargas Ríos     |            2
  E2024008 | Moreno Herrera  |            2
  E2024009 | Jiménez Ortiz   |            2
  E2024010 | Pardo Bernal    |            2
  E2024011 | Salazar Gómez   |            2
  E2024012 | Ortiz Jiménez   |            2
(12 rows)

Time: 5ms

PS C:\Users\Migatte\Desktop\Posgrado\BigDataNoSqlNewSql\practica-newsql-nosql>
```

Figura 6. Consultas 6 a 10: conteos, agregaciones por semestre, HAVING y subconsulta (A4).

Resultado observado. Estudiantes por programa: 5, 5, 5, 5 y 4 (Enfermería). Matrículas por estado: FINALIZADA 26, ACTIVA 6, APLAZADA 3, CANCELADA 1. Matrículas por semestre: 2024-1: 7; 2024-2: 12; 2025-1: 14; 2025-2: 3. Con `HAVING count(*) >= 8` aparecen Ingeniería de Sistemas y Administración de Empresas (8 cada una). La subconsulta devuelve 12 estudiantes con más matrículas que el promedio por estudiante.

Interpretación técnica. CockroachDB ejecuta SQL estándar con JOIN, `GROUP BY`, `HAVING` y subconsultas sin sintaxis especial; los resultados son consistentes con los datos cargados.

A5. Manipulación de datos

Objetivo. Demostrar UPDATE, DELETE y el comportamiento de las restricciones de integridad.

Procedimiento. `cockroachdb\05_manipulacion.sql`, ejecutado con `--set errexit=false` para que los errores esperados no detengan el script. El UPDATE cambia la ciudad de un estudiante y luego se restaura; el DELETE se aplica a un registro de prueba (`E9999999`) que se inserta para ese fin.

```
UPDATE estudiantes SET ciudad = 'Cajicá' WHERE codigo = 'E2024001';
DELETE FROM estudiantes WHERE codigo = 'E9999999';
```

Evidencia.

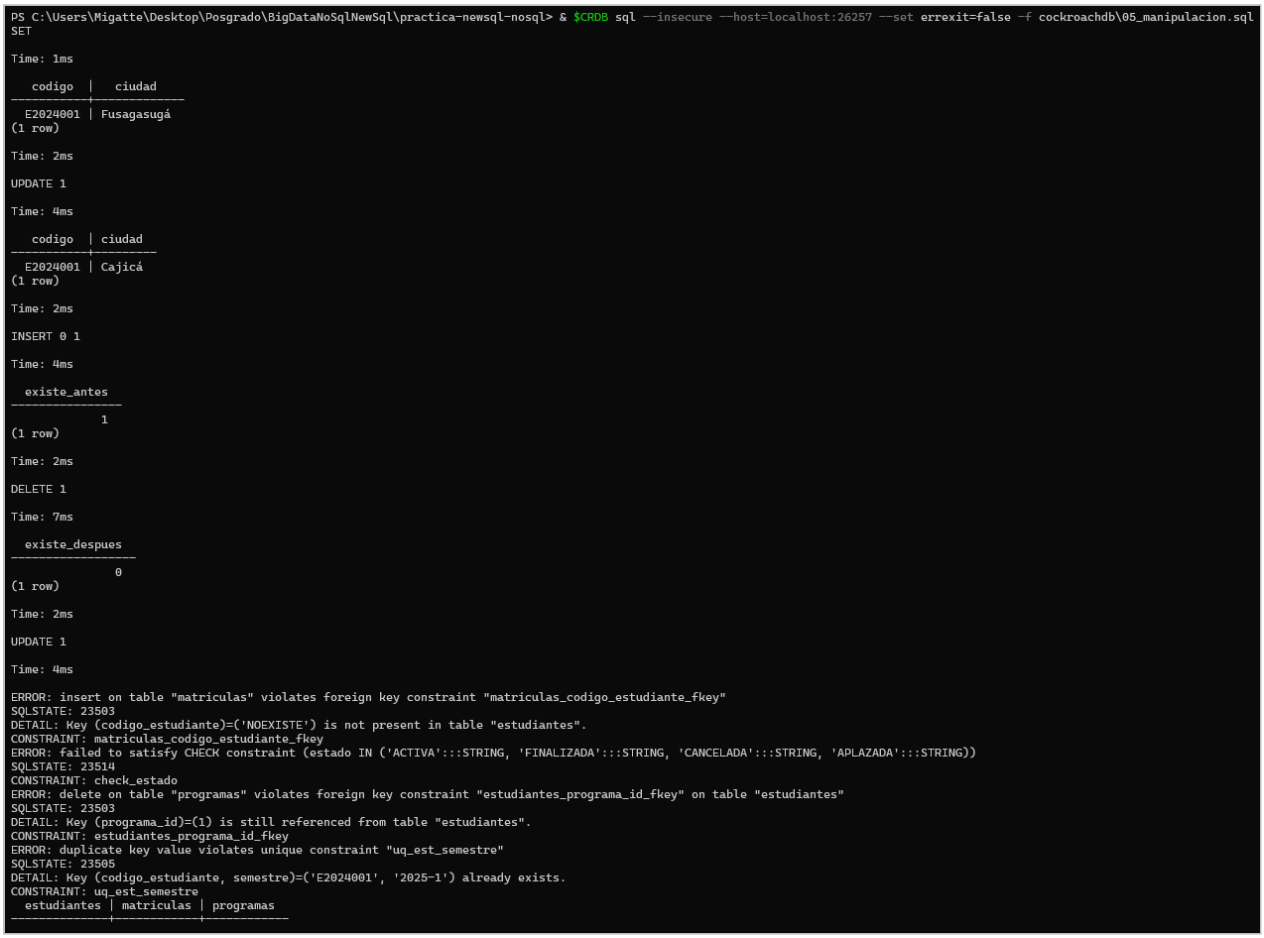


Figura 7. UPDATE, DELETE de un registro de prueba y violaciones de restricciones (A5).

Resultado observado. El UPDATE cambió la ciudad de Fusagasugá a Cajicá (validado con SELECT); el registro de prueba pasó de existir (1) a no existir (0). Las cuatro sentencias de prueba fueron rechazadas, como se esperaba: clave foránea al insertar (SQLSTATE 23503), CHECK de estado (23514), clave foránea al borrar un programa con estudiantes (23503) y unicidad de matrícula por semestre (23505). Los conteos finales no cambiaron (24, 36, 5).

Interpretación técnica. El motor protege la integridad referencial y de dominio; los datos inválidos nunca llegan a almacenarse. Este comportamiento contrasta con el observado en MongoDB (sección 9, B4).

A6. Transacciones

Objetivo. Demostrar `BEGIN`, `COMMIT` y `ROLLBACK`.

Procedimiento. `cockroachdb\06_transacciones.sql` ejecuta dos transacciones, cada una con dos inserciones (un estudiante de prueba y su matrícula): la primera termina con `COMMIT` y la segunda con `ROLLBACK`; al final se limpian los registros de prueba.

```
BEGIN;
INSERT INTO estudiantes (...) VALUES ('E9999997', ...);
INSERT INTO matriculas (...) VALUES ('E9999997', '2026-1', ...);
ROLLBACK;
```

Evidencia.

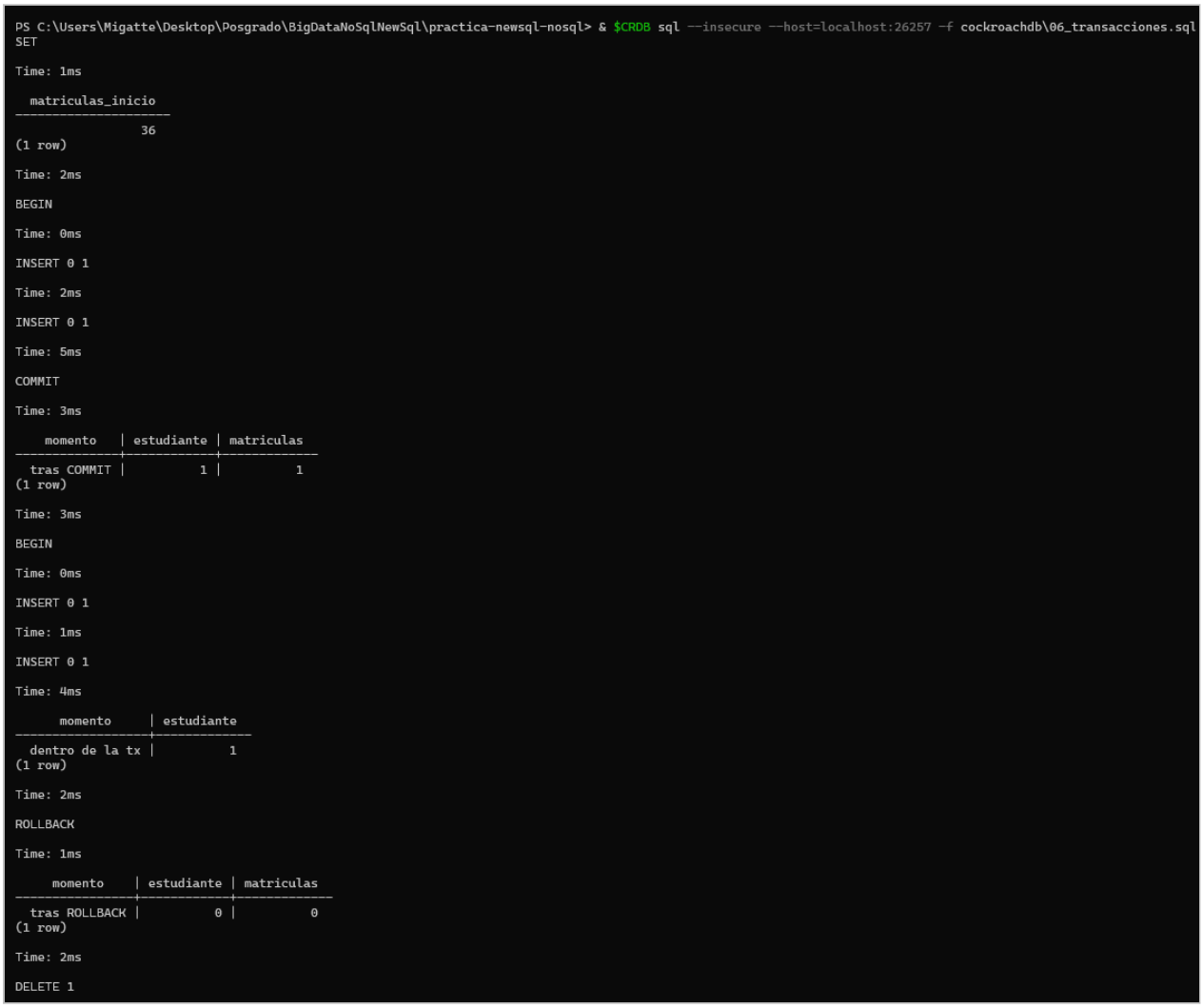


Figura 8. Transacciones con COMMIT y ROLLBACK (A6).

Resultado observado. Tras el `COMMIT`, el estudiante y su matrícula existen (1 y 1). Dentro de la segunda transacción el estudiante era visible (1); tras el `ROLLBACK` no existen ni el estudiante ni la matrícula (0 y 0). Después de la limpieza, las matrículas vuelven a 36 (igual al inicio).

Interpretación técnica. Se conservó la transacción confirmada y se revirtieron por completo los cambios de la abortada, lo que ilustra la atomicidad. Solo se probó una sesión única; no se evaluaron concurrencia, reintentos por conflictos ni aislamiento entre sesiones.

9. Desarrollo de la práctica con MongoDB

Todos los scripts se encuentran en `mongodb/` y se ejecutan desde la raíz del proyecto.

B1. Instalación y conexión

Objetivo. Instalar y verificar MongoDB Community Server y Compass, y conectar Compass a la instancia local.

Procedimiento. Se comprobó que no había instalaciones previas de MongoDB, mongosh ni Compass. Con autorización explícita se instalaron con `winget` los tres paquetes oficiales; el servicio `MongoDB` quedó en estado «Running» con inicio automático. En Compass se creó una conexión con la URI `mongodb://localhost:27017/`. `mongodb\01_verificacion.js` consulta la versión, hace `ping` y lista las bases.

Evidencia.

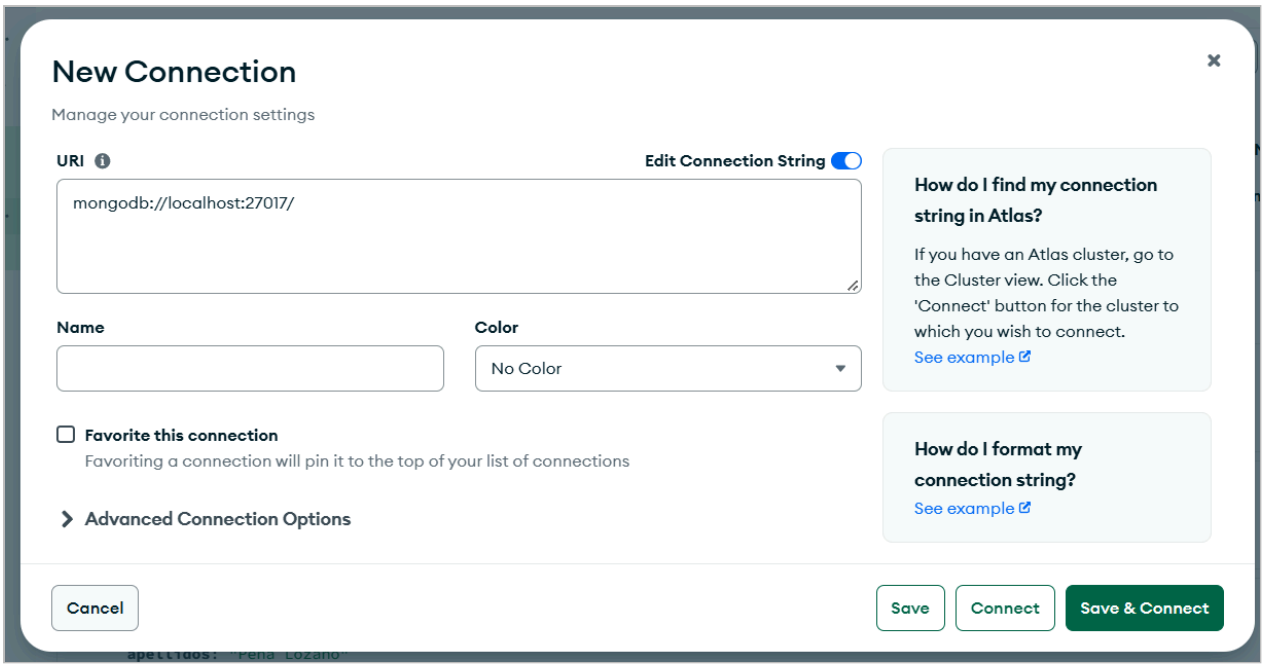


Figura 9. Cuadro de nueva conexión de Compass con la URI local (B1).

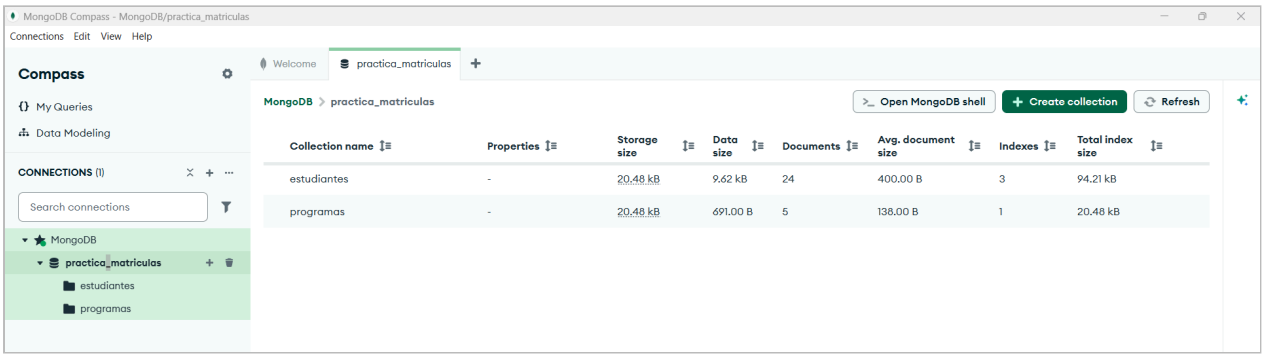


Figura 10. Compass conectado: base practica_matriculas y sus colecciones (B1 y B2).

Resultado observado. Servidor 9.0.2; `ping` devuelve `ok: 1`; el nodo es standalone (`isWritablePrimary: true`, sin conjunto de réplicas). Compass muestra la conexión activa y, tras la carga, la base con las colecciones `estudiantes` (24 documentos) y `programas` (5).

Interpretación técnica. Compass es un cliente gráfico y no incluye el servidor: ambos se instalaron por separado. La instancia es un único nodo sin réplicas.

B2. Estructura documental

Objetivo. Representar programas y matrículas con un modelo documental.

Procedimiento. `mongodb\02_carga_datos.js` lee `datos/programas.json` y `datos/estudiantes.json`, convierte las fechas a tipo `Date`, recrea las colecciones del proyecto y crea índices (único en `correo` y otro en `programa_id`).

Decisiones de modelado.

- **Colecciones y documentos:** `programas` (5 documentos) y `estudiantes` (24 documentos). Las matrículas se **embeben** como array dentro de cada estudiante, porque se consultan casi siempre junto con él y su cantidad por estudiante es pequeña (de una a dos). Cada estudiante guarda `programa_id` como referencia al programa.
- **Campos opcionales:** `telefono` existe solo en algunos documentos; en SQL es una columna `NULL`.
- **Tipos de datos:** cadenas, enteros, fechas BSON (`Date`) y arrays de subdocumentos.
- **Diferencias frente al esquema relacional:** 2 colecciones en lugar de 3 tablas; sin JOIN para leer un estudiante con sus matrículas; sin claves foráneas ni CHECK impuestos por el motor, ya que la integridad referencial queda a cargo de la aplicación.

Resultado observado. 5 programas, 24 estudiantes y 36 matrículas embebidas (conteo con `$unwind`), los mismos registros lógicos que en SQL (ver Figura 10).

B3. Inserción y consultas

Objetivo. Insertar y consultar documentos con filtros, proyecciones, ordenamiento, operadores y conteos.

Procedimiento. La inserción se hizo con `insertMany()` en `02_carga_datos.js`. `mongodb\03_consultas.js` contiene diez consultas: listado general, filtro por programa, proyección, ordenamiento, comparación (`$gte`), condiciones combinadas (`$and`/`$or`), campo opcional ausente (`$exists`), conteos, estudiantes con varias matrículas (`$expr + $size`) y búsqueda en arrays (`$elemMatch`).

```
db.estudiantes.find({ programa_id: 1 }, { nombres: 1, apellidos: 1 }).sort({ apellidos: 1 })
```

Evidencia.

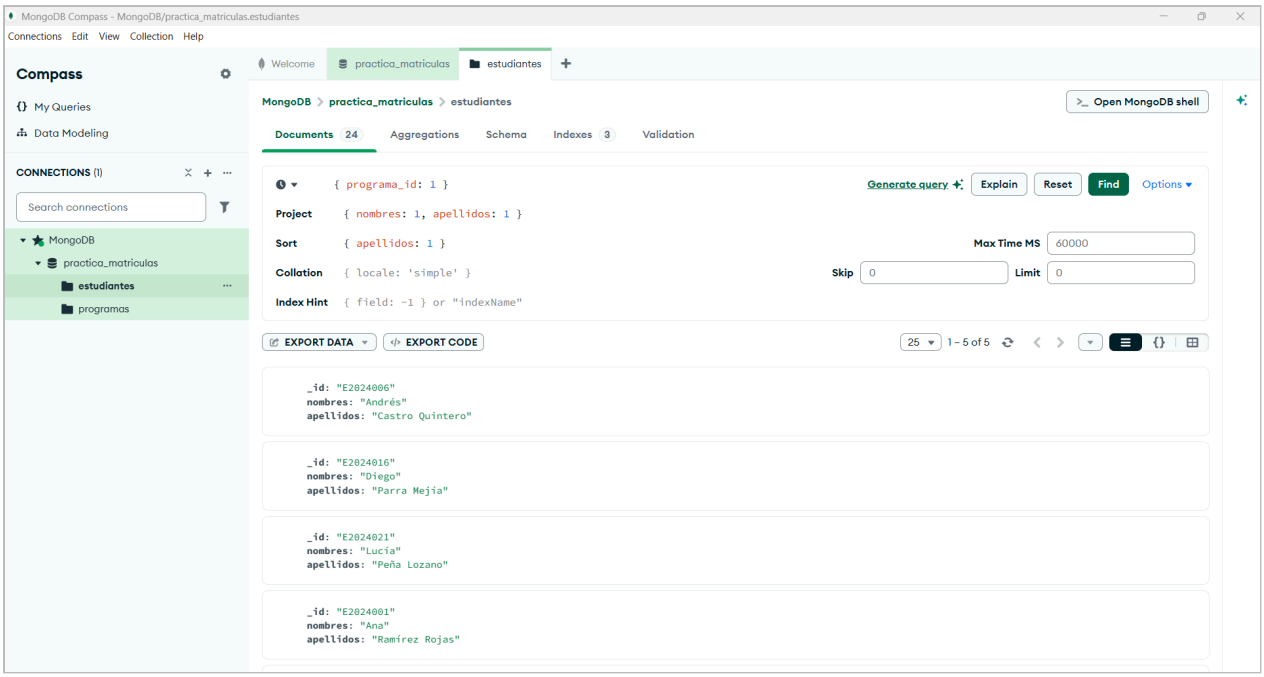


Figura 11. Filtro, proyección y ordenamiento en Compass (B3).

Resultado observado. En Compass, el filtro `{ programa_id: 1 }` con la proyección y el orden indicados devuelve 5 documentos (los mismos 5 estudiantes que en SQL). El script `03_consultas.js` (salida en `evidencias/salidas/mongo_03_consultas.txt`) produjo, entre otros: 24 estudiantes en total, 5 del programa 1, 5 programas, y un único estudiante con una matrícula CANCELADA (E2024023, igual que en la consulta SQL por estado).

Interpretación técnica. El lenguaje de consulta de MongoDB expresa filtros y proyecciones con documentos JSON. Gracias al embebido, consultar varias matrículas por estudiante no requiere uniones.

B4. Actualizaciones y eliminaciones

Objetivo. Demostrar `updateOne()`, `updateMany()` y `deleteOne()`, con comprobación de los cambios.

Procedimiento. `mongodb\04_actualizaciones.js` aplica: `updateOne` (ciudad de un estudiante, luego restaurada), `updateMany` con `$set` y `$unset` sobre los estudiantes del programa 5, una actualización de un elemento de un array con `arrayFilters` (luego restaurada), `deleteOne` sobre un documento de prueba y un insert sin validación para comparar con SQL. No se hace ningún borrado masivo.

```
db.estudiantes.updateMany({ programa_id: 5 }, { $set: { etiqueta: "revisado" } })
db.estudiantes.deleteOne({ _id: "E9999999" })
```

Evidencia.

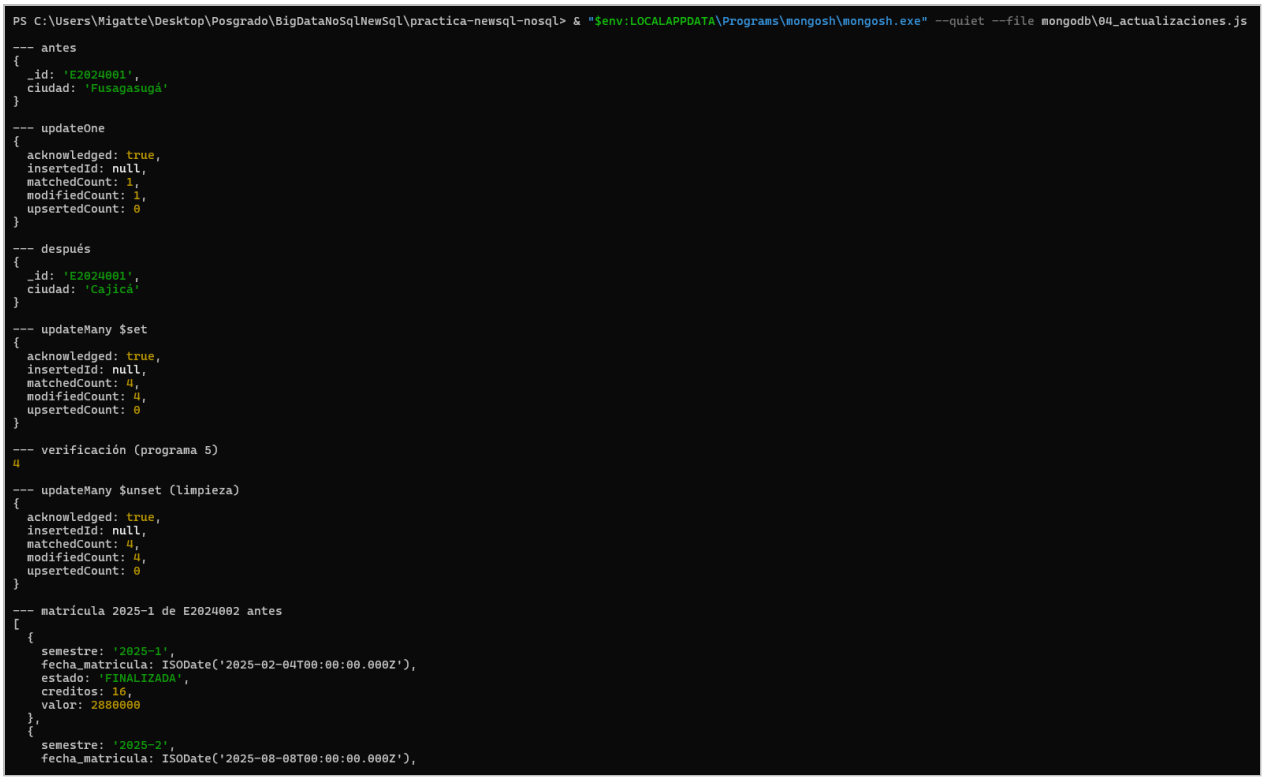


Figura 12. updateOne y updateMany (B4).

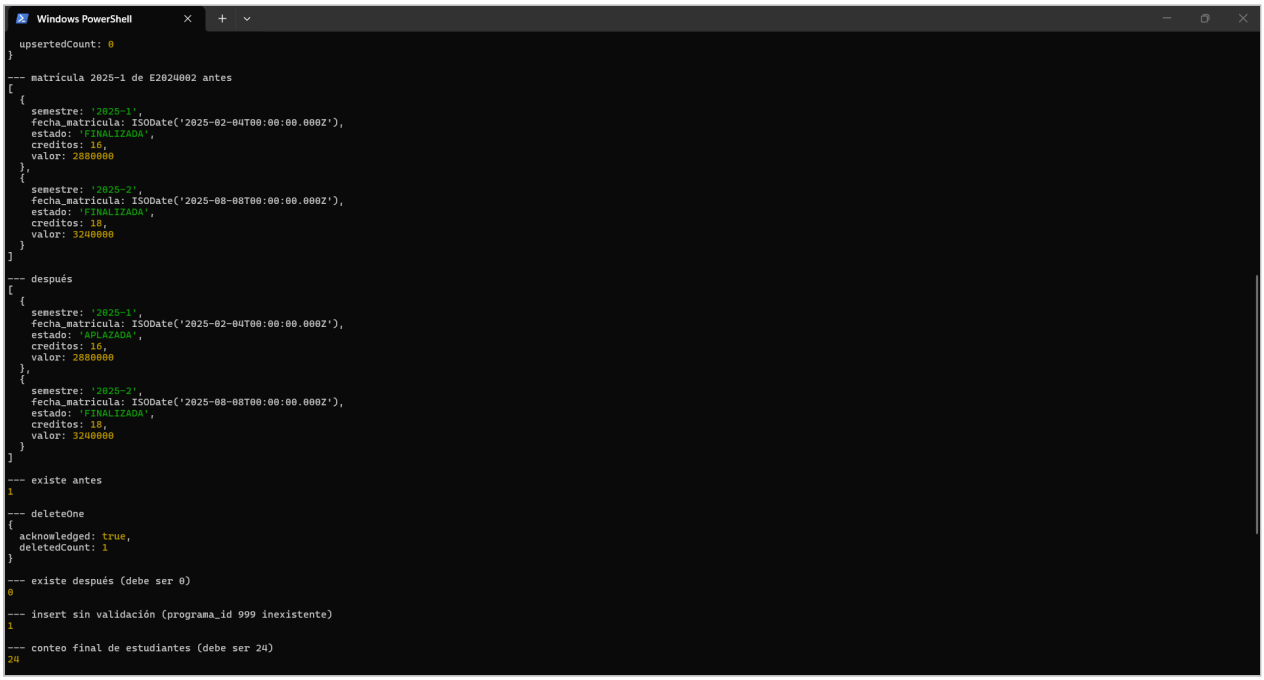


Figura 13. Actualización en array con arrayFilters, deleteOne y conteo final (B4).

Resultado observado. `updateOne`: `matchedCount: 1`, `modifiedCount: 1` (Fusagasugá → Cajicá). `updateMany`: 4 documentos coincidentes y modificados (programa 5), verificados y luego limpiados con `$unset`. El estado de la matrícula 2025-1 de E2024002 cambió de FINALIZADA a APLAZADA y se restauró. `deleteOne`: `deletedCount: 1` y el conteo del documento de prueba pasó de 1 a 0. El conteo final de estudiantes es 24. Además, MongoDB **aceptó** un documento de prueba con `programa_id: 999` (inexistente) y un estado inválido, que luego se eliminó.

Interpretación técnica. Sin una validación de esquema configurada, MongoDB no rechazó datos que SQL sí rechazó (compárese con A5). Esto es una diferencia observada en las condiciones de esta práctica: MongoDB permite definir validación de esquema, pero no se configuró aquí.

B5. Agregaciones

Objetivo. Obtener información resumida mediante pipelines de agregación.

Procedimiento. `mongodb\05_agregaciones.js` define cuatro pipelines:

1. **Estudiantes por programa:** `$group` (agrupa y cuenta), `$lookup` (equivale al JOIN con `programas`), `$unwind`, `$project` y `$sort`.
2. **Matrículas por estado:** `$unwind` (un documento por matrícula embebida), `$group` y `$sort`.
3. **Resumen por semestre:** `$unwind`, `$group` con `$sum` de créditos y valor, `$sort`.
4. **Programas con al menos 8 matrículas:** `$group` seguido de `$match` (equivalente a `HAVING`) y `$lookup`.

Evidencia.

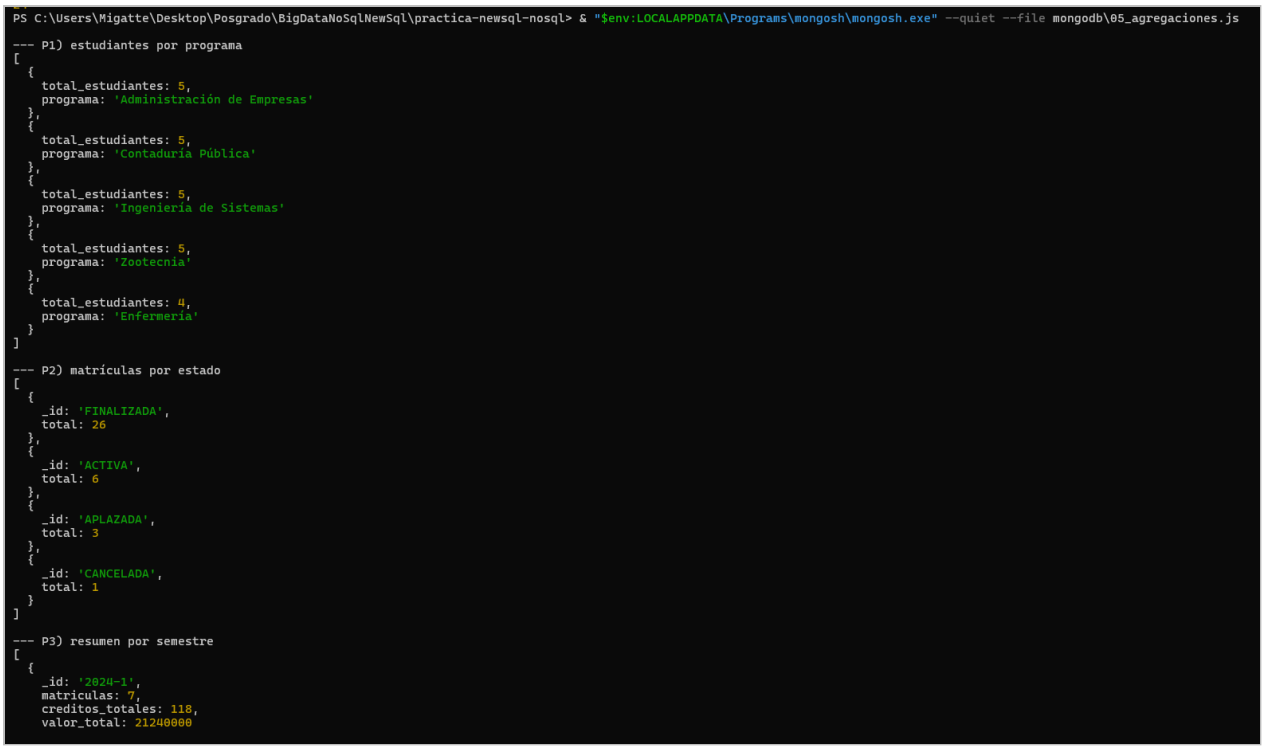


Figura 14. Pipelines 1 y 2 y el inicio del 3 (B5).



Figura 15. Pipelines 3 y 4 (B5).

Resultado observado. Estudiantes por programa: 5, 5, 5, 5 y 4; matrículas por estado: FINALIZADA 26, ACTIVA 6, APLAZADA 3, CANCELADA 1; resumen por semestre: 7, 12, 14 y 3 matrículas (2024-1 a 2025-2); programas con al menos 8 matrículas: Administración de Empresas e Ingeniería de Sistemas, con 8 cada uno. Todos coinciden con los resultados SQL de A4.

Interpretación técnica. El pipeline de agregación cumple el rol de `GROUP BY`/`HAVING`/`JOIN` de SQL, en etapas encadenadas. `$unwind` es la operación clave para agregar sobre matrículas embebidas.

B6. Exportación y comprobación

Objetivo. Documentar la exportación de datos o resultados con Compass y comprobar la reproducibilidad.

Procedimiento previsto. En Compass, pestaña *Documents*, botón **Export Data** (visible en la Figura 11) → *Export the full collection* o *Export query results* → formato JSON o CSV. El procedimiento está en el `README.md`.

Resultado observado. La exportación desde Compass no se ejecutó, por lo que no hay evidencia de ella en este informe. En cuanto a la reproducibilidad, los datos de partida se conservan en el repositorio (`datos/*.json`), y los scripts de carga y consulta se ejecutaron correctamente en ambos motores. No se hizo una prueba formal desde un equipo limpio: solo se repitió la carga sobre el mismo entorno.

10. Resultados observados

Verificación	CockroachDB	MongoDB
Programas / estudiantes / matrículas	5 / 24 / 36	5 / 24 / 36 (embebidas)
Estudiantes por programa	5, 5, 5, 5, 4	5, 5, 5, 5, 4
Matrículas por estado	FINALIZADA 26, ACTIVA 6, APLAZADA 3, CANCELADA 1	Igual
Matrículas por semestre	7, 12, 14, 3	Igual
Programas con ≥ 8 matrículas	Sistemas y Administración (8 c/u)	Igual
Dato inválido (estado o referencia)	Rechazado por FK/CHECK	Aceptado (sin validación configurada)
Transacción con `COMMIT/ROLLBACK`	Probada	No probada

Las diferencias prácticas observadas se limitan a lo indicado en la tabla y a la forma de expresar las consultas (JOIN y `GROUP BY` frente a `$lookup` y pipelines).

11. Comparación técnica

Los puntos marcados «documentación» provienen de las fuentes citadas y no fueron comprobados en la práctica; los marcados «observado» sí se comprobaron.

Aspecto	CockroachDB	MongoDB
Modelo de datos	Relacional distribuido, con SQL (documentación: Taft et al., 2020). *Observado:* 3 tablas	Documental (documentación: MongoDB, Inc., s. f.-b). *Observado:* 2 colecciones con matrículas embebidas

Aspecto	CockroachDB	MongoDB
Esquema e integridad	*Observado:* esquema fijo; FK, CHECK y UNIQUE rechazaron datos inválidos	*Observado:* sin esquema impuesto por defecto; aceptó datos inválidos. El motor admite validación, que no se configuró
Relaciones	*Observado:* JOIN y claves foráneas	*Observado:* embebido y referencia por `programa_id`; `$lookup` para unir
Consultas y agregaciones	*Observado:* SQL con `GROUP BY` y `HAVING`	*Observado:* pipelines (`$group`, `$unwind`, `$lookup`) con los mismos resultados
Transacciones	*Observado:* `BEGIN/COMMIT/ROLLBACK` en un nodo. Documentación: aislamiento SERIALIZABLE por defecto (Cockroach Labs, s. f.-a)	*No probadas.* Documentación: el manual indica que el soporte de transacciones distribuidas no sustituye un buen diseño de esquema (MongoDB, Inc., s. f.-b)
Escalabilidad y distribución	Documentación: diseñado para distribuirse (Taft et al., 2020). *No probada:* un solo nodo	Documentación: pensado para cargas distribuidas (Cattell, 2010). *No probada:* instancia standalone
Flexibilidad	Cambios de esquema requieren `ALTER`	*Observado:* `telefono` ausente en algunos documentos; `etiqueta` añadida y retirada sin cambiar esquema
Casos de uso apropiados	Documentación: cargas transaccionales (OLTP) con consistencia fuerte (Taft et al., 2020)	Documentación: datos con estructura variable y lecturas centradas en un documento (MongoDB, Inc., s. f.-b)

Rendimiento. No se realizaron pruebas controladas ni mediciones, por lo que no se afirma que un motor sea más rápido que el otro. Tampoco debe interpretarse el uso local de ambos motores como una prueba de escalabilidad.

12. Limitaciones

- Ambos motores se ejecutaron en un único nodo local; no se probó distribución, alta disponibilidad, replicación ni escalabilidad.
- CockroachDB funcionó en modo inseguro, de desarrollo.
- El conjunto de datos es pequeño y sintético; no permite conclusiones de rendimiento.
- No se midieron tiempos ni se hicieron comparaciones de velocidad.
- Las transacciones solo se probaron en CockroachDB y en una única sesión.
- La exportación desde Compass (B6) no se ejecutó, ni se hizo una prueba desde un entorno limpio.
- Los ejercicios son de diseño propio, no oficiales del docente.

13. Conclusiones

1. Se verificó y utilizó CockroachDB v26.3.2 en un nodo local, con un esquema relacional de tres tablas, carga repetible de 5 programas, 24 estudiantes y 36 matrículas, consultas, manipulación y transacciones ejecutadas con resultados comprobados.

2. Se instaló y verificó MongoDB 9.0.2 con mongosh y Compass; el mismo caso se modeló con dos colecciones y matrículas embebidas, y las consultas, actualizaciones y agregaciones se ejecutaron correctamente.
3. Los resultados agregados coincidieron entre ambos motores, lo que confirma que los registros lógicos son equivalentes.
4. La diferencia práctica más clara fue la integridad: en CockroachDB las restricciones rechazaron datos inválidos; en MongoDB, sin validación configurada, esos datos se aceptaron.
5. Las afirmaciones sobre escalabilidad, alta disponibilidad, rendimiento y transacciones en MongoDB provienen de la documentación o quedan sin probar; para sustentarlas haría falta un entorno con varios nodos y mediciones controladas.

14. Referencias

Cattell, R. (2010). Scalable SQL and NoSQL data stores. *ACM SIGMOD Record, 39*(4), 12–27. https://sigmodrecord.org/publications/sigmodRecord/1012/pdfs/04.surveys.cattell.pdf

Cockroach Labs. (s. f.-a). *Transactions*. CockroachDB Docs. https://www.cockroachlabs.com/docs/stable/transactions

Cockroach Labs. (s. f.-b). *BEGIN*. CockroachDB Docs. https://www.cockroachlabs.com/docs/stable/begin-transaction

MongoDB, Inc. (s. f.-a). *Aggregation operations*. MongoDB Manual. https://www.mongodb.com/docs/manual/aggregation/

MongoDB, Inc. (s. f.-b). *aggregate (database command)*. MongoDB Manual. https://www.mongodb.com/docs/manual/reference/command/aggregate/

NIST Big Data Public Working Group. (2015). *NIST big data interoperability framework: Volume 1, definitions* (NIST Special Publication 1500-1). National Institute of Standards and Technology. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.1500-1.pdf

Pavlo, A., y Aslett, M. (2016). What's really new with NewSQL? *ACM SIGMOD Record, 45*(2), 45–55. https://sigmodrecord.org/publications/sigmodRecord/1606/pdfs/07_forum_Pavlo.pdf

Taft, R., Sharif, I., Matei, A., VanBenschoten, N., Lewis, J., Grieger, T., Niemi, K., Woods, A., Birzin, A., Poss, R., Bardea, P., Ranade, A., Darnell, B., Gruneir, B., Jaffray, J., Zhang, L., y Mattis, P. (2020). CockroachDB: The resilient geo-distributed SQL database. En *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. https://doi.org/10.1145/3318464.3386134

15. Repositorio GitHub

https://github.com/Migatte-Yk/NoSqlNewSqlBigData

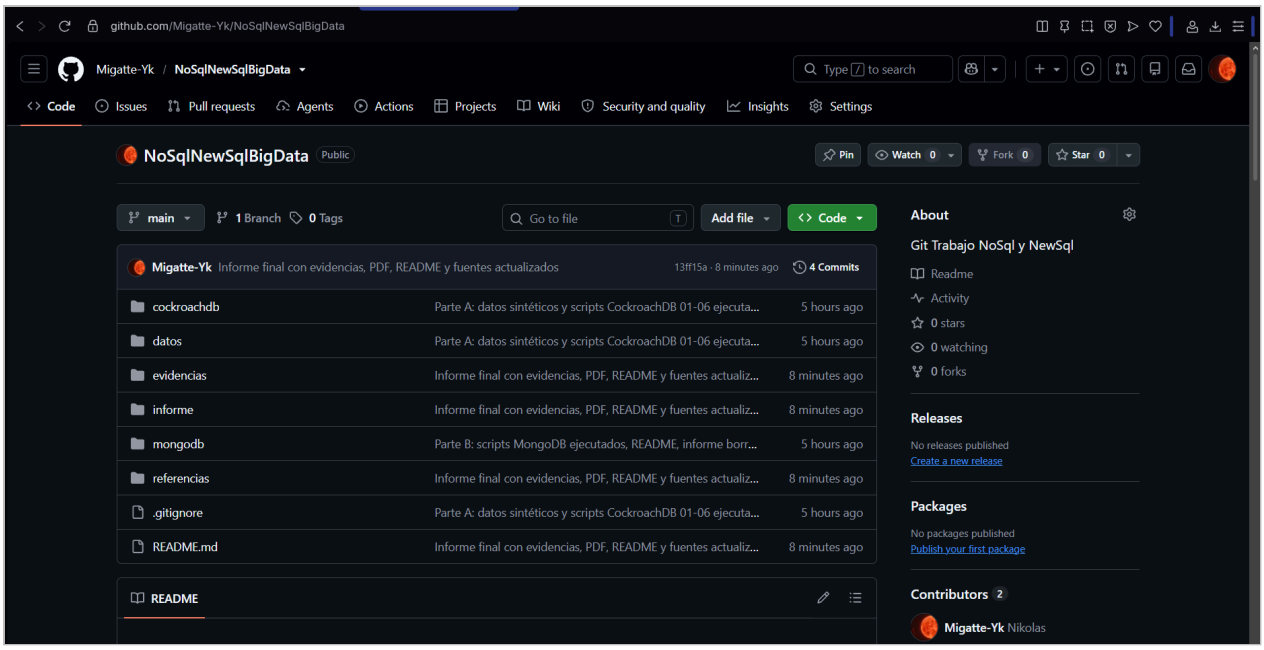


Figura 16. Repositorio en GitHub.